Lateralus Language Specification v3.0

Complete Language Reference: Grammar, Types, Semantics, and Standard Library

Lateralus Language

bad-antics · April 2026 · Lateralus Language Research

ABSTRACT Lateralus Language -- lateralus.dev

1. Language Overview and Goals

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

2. Notation Used in This Specification

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

3. Lexical Structure: Characters and Encoding

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

4. Lexical Structure: Whitespace and Comments

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
```

```
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                    // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

5. Lexical Structure: Identifiers

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

6. Lexical Structure: Keywords

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

Normative Definition

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {

    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

7. Lexical Structure: Literals: Integers

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Normative Definition

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports

circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

8. Lexical Structure: Literals: Floats

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Normative Definition

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

9. Lexical Structure: Literals: Strings

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct

construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

Normative Definition

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

10. Lexical Structure: Literals: Booleans and Nil

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is

LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Normative Definition

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

11. Lexical Structure: Operators and Punctuation

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions

(fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Normative Definition

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

12. Grammar: Top-Level Structure (EBNF)

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Normative Definition

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

13. Grammar: Declarations

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

14. Grammar: Statements

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete

type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

15. Grammar: Expressions

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

16. Grammar: Patterns

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
```

```
    fn zero() -> int { return 0 }

    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

17. Grammar: Types

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a

work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",

        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

18. Grammar: Operator Precedence Table

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

Normative Definition

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {

    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

19. Semantics: Values and Types

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Normative Definition

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files,

breaking cycles by giving each module a declared interface before its implementation is compiled.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

20. Semantics: Variables and Bindings

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import

statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Normative Definition

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it
```

```
fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

21. Semantics: Mutation (let mut)

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions

(fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

Normative Definition

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

22. Semantics: Scope and Shadowing

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Normative Definition

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

23. Semantics: Closures

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary

operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Normative Definition

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

24. Semantics: Function Calls and Arguments

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type

alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Normative Definition

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows

may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

25. Semantics: Control Flow: if/else

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by

the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope,

the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

26. Semantics: Control Flow: while

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
```

```
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

27. Semantics: Control Flow: for-in

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics)

at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

28. Semantics: Control Flow: match

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

29. Semantics: Control Flow: return/break/continue

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module

provides async/await and the task scheduler.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,

        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

30. Type System: Primitive Types

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides

manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

Normative Definition

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self

    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

31. Type System: Composite Types (struct, enum)

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Normative Definition

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files,

breaking cycles by giving each module a declared interface before its implementation is compiled.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
```

```
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

32. Type System: Generic Types

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Normative Definition

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors),

and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

33. Type System: Trait Bounds

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

Normative Definition

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

34. Type System: Type Inference (Algorithm W)

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent

explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Normative Definition

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it
```

```
fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

35. Type System: Unification

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if

expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Normative Definition

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

36. Type System: Occurs Check

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies

that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Normative Definition

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
```

```
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

37. Type System: Let Polymorphism

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary

operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait

implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

38. Type System: Subtyping and Coercion

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

39. Memory Model: Ownership

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Pipeline with types at each stage
```

```
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

40. Memory Model: Borrowing

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants

are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

41. Memory Model: Lifetimes

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

42. Memory Model: Move Semantics

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

Normative Definition

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,

}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

43. Memory Model: Copy Trait

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Normative Definition

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },

    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

44. Collections: list[T]

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Normative Definition

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the

normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

45. Collections: map[K,V]

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

Normative Definition

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

46. Collections: set[T]

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Normative Definition

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

47. String Operations

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are

exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Normative Definition

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return

type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

48. Error Handling: Option[T]

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent

explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Normative Definition

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

49. Error Handling: Result[T,E]

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all

expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e),

Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

50. Error Handling: Propagation with ?

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

51. Pipeline Operator: Semantics

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

52. Pipeline Operator: Type Inference

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness

checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

53. Pipeline Operator: Fusion Rules

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

54. Pattern Matching: Exhaustiveness

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

Normative Definition

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is

executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }

}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

55. Pattern Matching: Guard Clauses

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Normative Definition

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)

    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

56. Traits: Declaration

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Normative Definition

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the

normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

57. Traits: Implementation

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

Normative Definition

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

58. Traits: Default Methods

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent

explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Normative Definition

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
```

```
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

59. Traits: Standard Traits (Eq, Ord, Hash, Display)

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Normative Definition

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

60. Module System: Declarations and Imports

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent

explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Normative Definition

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

61. Module System: Visibility

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all

expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e),

Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

62. Module System: Circular Dependencies

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

63. Async/Await: Task Model

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

64. Async/Await: The Event Loop

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for

fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

65. Foreign Function Interface (FFI)

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {

    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
```

```
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

66. Macros

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

Normative Definition

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",

        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

67. Standard Library: Overview

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Normative Definition

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums

        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

68. Standard Library: io Module

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Normative Definition

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

69. Standard Library: net Module

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is

guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

Normative Definition

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is

'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it

fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

70. Standard Library: crypto Module

Modules in Lateralus define the unit of compilation and the unit of capability declaration. Each source file is a module. A module begins with an optional 'module ModuleName' declaration followed by import statements and definitions. The 'pub' visibility modifier makes a definition accessible from other modules. By default, all definitions are private to the module. Re-exports allow a module to make imported names part of its public interface. The module system supports circular imports through interface files (.lti): the compiler processes interface files before processing source files, breaking cycles by giving each module a declared interface before its implementation is compiled.

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V],

Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Normative Definition

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
```

```
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

71. Standard Library: math Module

The Lateralus programming language is a statically typed, expression-oriented language with first-class pipeline operators, algebraic data types, and a Hindley-Milner type inference engine. Its design is guided by three principles: clarity (code should read as naturally as prose), correctness (the type system should prevent entire classes of errors), and performance (zero-cost abstractions that compile to efficient machine code). This specification defines the normative syntax and semantics of the Lateralus language version 3.0, superseding previous specifications.

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Normative Definition

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

```
// Enum with data variants
pub enum Shape {
    Circle { center: Point, radius: float },
    Rect   { origin: Point, width: float, height: float },
    Triangle { a: Point, b: Point, c: Point },
}

pub fn area(s: Shape) -> float {
    match s {
        Shape.Circle { radius: r, .. }       => math.PI * r * r,
        Shape.Rect   { width: w, height: h, .. } => w * h,
        Shape.Triangle { a, b, c }           => triangle_area(a, b, c),
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

72. Standard Library: strings Module

The Lateralus grammar is specified as an Extended Backus-Naur Form (EBNF) grammar. The top-level production is 'module', which consists of a module declaration, zero or more import statements, and a sequence of definitions. Definitions include function definitions, struct definitions, enum definitions, trait definitions, implementation blocks, type alias definitions, and constant declarations. The grammar is LL(1) for the statement level and uses Pratt parsing for expressions, providing unambiguous precedence climbing without explicit grammar rules for each precedence level.

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler

checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Normative Definition

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

```
// Generic function with trait bound
trait Summable {
    fn zero() -> Self
    fn add(self, other: Self) -> Self
}

fn sum_all[T: Summable](items: list[T]) -> T {
    return items |> reduce(T.zero(), fn(acc, x) { acc.add(x) })
}

impl Summable for int {
    fn zero() -> int { return 0 }
    fn add(self, other: int) -> int { return self + other }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

73. Standard Library: collections Module

Expressions are the fundamental units of value computation in Lateralus. Every expression has a static type, inferred by the type checker. The expression forms are: literals (integer, floating-point, string, boolean, nil), identifiers, binary operations, unary operations, pipeline operations (|>), function calls, method calls, index expressions, struct

construction, enum variant construction, match expressions, if expressions, block expressions, lambda expressions (fn(...) { ... }), and type cast expressions (value as Type). Function calls, match expressions, and blocks are all expressions that produce values.

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Normative Definition

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e),

Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

```
// Pattern matching -- exhaustiveness
fn describe_result(r: Result[int, str]) -> str {
    match r {
        Ok(n) if n > 100 => "large positive",
        Ok(n) if n > 0   => "small positive",
        Ok(0)            => "zero",
        Ok(n)            => "negative: " + str(n),
        Err(e)           => "error: " + e,
        // All cases covered -- no warning
    }
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

74. Standard Library: testing Module

Type annotations in Lateralus are optional but checked. When a programmer writes 'let x: int = 5', the compiler verifies that the inferred type of '5' is compatible with 'int'. When a function declares a return type 'fn foo() -> str', the compiler checks that all return paths produce a 'str'. Annotations are required in two situations: function parameters (to support separate compilation without full-program inference) and cases where the programmer wants to document intent explicitly. The type language includes: primitive types (int, float, str, bool, u8, u64), generic types (list[T], map[K,V], Option[T], Result[T,E]), function types (fn(A,B)->C), trait objects (dyn Trait), and user-defined types.

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

Normative Definition

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

```
// Pipeline with types at each stage
fn pipeline_example(nums: list[int]) -> map {
    let analysis = nums
        |> filter(fn(x) { x > 0 })                    // list[int]
        |> map(fn(x: int) -> float { sqrt(x as float) }) // list[float]
        |> take(10)                                     // list[float]
        |> fold({ "sum": 0.0, "count": 0 }, fn(acc, x) {
            acc["sum"]   = acc["sum"] + x
            acc["count"] = acc["count"] + 1
            return acc
        })  // map
    return analysis
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

75. Compiler: Compilation Model

The ownership system tracks the lifetime of each value and ensures that values are not used after they have been freed or transferred. Every value has exactly one owner. When the owner goes out of scope, the value is freed. Values can be moved (transferring ownership) or borrowed (lending a reference without transferring ownership). Mutable borrows are exclusive: while a mutable borrow is active, no other borrows of the same value may exist. Immutable borrows are shared: multiple immutable borrows may coexist. The borrow checker, implemented as a dataflow analysis over the control flow graph, verifies that all borrows obey these rules at compile time.

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

Normative Definition

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

```
// Async function and await
async fn fetch_and_parse(url: str) -> Result[Data, str] {
    let response = await http.get(url)
    let text     = await response.text()
    match parse_json(text) {
        Ok(json) => Ok(Data.from_json(json)),
        Err(e)   => Err("Parse error: " + e),
    }
}

async fn main() {
```

```
    let tasks = URLS |> map(fn(url) { fetch_and_parse(url) })
    let results = await gather(tasks)   // concurrent execution
    results |> filter(fn(r) { r.is_ok() }) |> for_each(println)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

76. Compiler: Error Messages

Pattern matching in Lateralus provides exhaustive, structured destructuring. The 'match' expression takes a value and a sequence of arms, each consisting of a pattern and a result expression. Patterns can be: wildcard (_), variable binding (x), literal (42, "hello", true), enum variant (Some(x), Err(e), Color.Red), struct pattern ({x, y, z}), list pattern ([h, ...t]), tuple pattern ((a, b)), and guard pattern (pattern if condition). The compiler verifies that match expressions are exhaustive: every possible value of the matched type is covered by at least one arm. Missing patterns are compile errors with a suggestion of which cases are unhandled.

Enums in Lateralus are algebraic data types -- tagged unions whose variants can carry data. An enum variant can hold any number of fields (unnamed tuple variants or named struct variants). The canonical pattern for nullable values is 'Option[T]' with variants 'Some(T)' and 'None'. The canonical pattern for fallible operations is 'Result[T,E]' with variants 'Ok(T)' and 'Err(E)'. The compiler's exhaustiveness checker ensures that all code paths handle all variants, preventing the class of bugs where new variants are added to an enum but some match expressions are not updated to handle them.

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides

cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

Normative Definition

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

```
// Ownership and borrowing
fn process(data: list[int]) -> int {
    let total = sum(data)     // borrows data (immutable)
    let mean  = total / len(data)  // data still accessible
    return mean
}
// data is NOT moved -- process only borrows it
```

```
fn move_example() {
    let v = [1, 2, 3]
    let w = v               // v is MOVED into w
    // println(v)           // ERROR: v was moved
    println(w)              // OK
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

77. Grammar Appendix: Complete EBNF

Traits define interfaces that types can implement. A trait declares a set of method signatures; implementing a trait requires providing a concrete implementation of each method. Traits support default method implementations, which can be overridden by implementations. A type implements a trait by providing an 'impl TraitName for TypeName' block. Trait bounds constrain type parameters: 'fn foo[T: Ord + Hash](x: T)' requires that 'T' implement both 'Ord' and 'Hash'. The compiler verifies trait implementations at the use site: if a function requires 'T: Ord', every call to that function with a concrete type 'T' is checked to ensure 'T' implements 'Ord'.

Generics allow functions and types to be parameterized over types. A generic function 'fn id[T](x: T) -> T { return x }' works for any type T. A generic struct 'struct Pair[A, B] { first: A, second: B }' holds two values of potentially different types. Generic parameters can be constrained with trait bounds. Lateralus uses monomorphization: for each distinct set of type arguments, the compiler generates a specialized copy of the generic function or type. This produces efficient code (no boxing, no dynamic dispatch for generics) at the cost of potentially larger binary sizes.

The Lateralus execution model supports both compiled (static) and interpreted (VM) execution. In compiled mode, the compiler generates Python, C, RISC-V, or x86-64 assembly, which is then passed to the host toolchain for final compilation. In VM mode, the compiler generates LBC bytecode, which is executed by the Lateralus VM. The VM supports JIT compilation: hot functions are detected by a sampling profiler and recompiled to machine code at runtime. The async runtime is built on a work-stealing task queue: async functions return immediately, scheduling a continuation to be executed when the awaited value is ready.

Normative Definition

The standard library modules provide the building blocks of Lateralus programs. The 'collections' module provides list[T], map[K,V], set[T], deque[T], and heap[T]. The 'strings' module provides manipulation functions: split, join, trim, replace, starts_with, ends_with, contains, format. The 'io' module provides file I/O and console I/O. The 'net' module provides TCP/UDP networking. The 'math' module provides arithmetic functions, trigonometry, and numerical algorithms. The 'crypto' module provides cryptographic primitives (AES-GCM, ChaCha20-Poly1305, SHA-256, Argon2). The 'async' module provides async/await and the task scheduler.

```
// Module structure
module geometry

import math

pub struct Point {
    x: float,
    y: float,
}

pub fn distance(a: Point, b: Point) -> float {
    let dx = b.x - a.x
    let dy = b.y - a.y
    return math.sqrt(dx*dx + dy*dy)
}
```

Keyword Kind Since Purpose

fn declaration v1.0 function definition

let statement v1.0 variable binding

mut modifier v1.0 mutable variable

pub modifier v1.0 public visibility

struct declaration v1.0 product type

enum declaration v1.0 sum type

trait declaration v1.0 interface

impl declaration v1.0 implementation

match expression v1.0 pattern matching

|> operator v1.0 pipeline forward

async/await expression v1.3 concurrency

guard statement v1.4 early return guard

measure statement v1.4 performance profiling

emit statement v1.4 event tracing

from...import statement v1.5 selective import

Lateralus is an open-source, zero-dependency programming language. Project home: https://lateralus.dev. Source: github.com/bad-antics/lateralus-lang. Released under CC BY 4.0.